

Programspråk

Kapitel 10: Kontrollstrukturer och subprogram

Henrik Björklund



UMEÅ UNIVERSITY

Imperativa språk består huvudsakligen av instruktioner av typen:

```
var = expr;
```

Imperativa språk består huvudsakligen av instruktioner av typen:

```
var = expr;
```

För att få ett **Turing-fullständigt** språk behöver vi två saker till:

- Villkor (enklaste varianten: **if**)
- Hopp (enklaste - och kraftfullaste - varianten: **goto**)

- Nästlade if-then-else
 - Läsbarhet! Använd alltid block
 - Vissa språk har någon form av “elif”
- Flera val
 - switch-case
 - Finns i många språk men inte alla (t.ex. Python)
 - Med/utan “break” ... ger olika programmeringsstil
 - Default?
 - Pattern matching

- Logiska loopar
 - while (expr) do ... end
 - do ... while (expr)
- Kontrollera loopar
 - break
 - continue

Iteration

```
while (uttryck){ ...  
    if (uttryck){  
        break;  
    }  
    ...  
}
```

```
while (uttryck){  
    ...  
    if (uttryck){  
        continue;  
    }  
    ...  
}
```

- Iteratorer i stället för räknare
- Smidigt och flexibelt
- Kan vara listor, träd, etc.

```
for x in range(4):  
    print x
```

```
fruit = ['apelsin', 'banan', 'citron']
```

```
for x in range(len(fruit)):  
    print fruit[x]
```

```
for x in fruit:  
    print x
```

Guarded commands

```
if <bool exp> -> <statements>  
[] <bool exp> -> <statements>  
...  
[] <bool exp> -> <statements>  
fi
```

```
do <bool exp> -> <statements>  
[] <bool exp> -> <statements>  
...  
[] <bool exp> -> <statements>  
od
```

```
do q1>q2 -> tmp=q1; q1=q2; q2=tmp;  
[] q2>q3 -> tmp=q2; q2=q3; q3=tmp;  
[] q3>q4 -> tmp=q3; q3=q4; q4=tmp;  
od
```

Abstraktion behövs för att det är så vi människor fungerar.

Abstraktion behövs för att det är så vi människor fungerar.

- Inkapsling (encapsulation)
 - Organiserar koden (lättare att förstå)
 - Lättare att ändra och underhålla
 - Separatkompilering
- Information hiding
 - Pålitlighet - Kod som inte är "betrodd" kan inte komma åt eller ändra "intern" information

```
MODULE PrintHelloWorld;  
  
(*This program prints "Hello world!" on the standard output device*)  
  
FROM InOut IMPORT WriteString, WriteLn;  
  
BEGIN  
  WriteString("Hello world!");  
  WriteLn;  
END PrintHelloWorld.
```

```
DEFINITION MODULE Circles;
```

```
PROCEDURE AreaOfCircle(Radius : REAL; VAR Area : REAL);
```

```
PROCEDURE PerimeterOfCircle(Radius : REAL; VAR Perim : REAL);
```

```
END Circles.
```

```
IMPLEMENTATION MODULE Circles;
```

```
PROCEDURE GetPi(VAR Pi : REAL);
```

```
BEGIN
```

```
  Pi := 3.14159;
```

```
END GetPi;
```

```
PROCEDURE AreaOfCircle(Radius : REAL; VAR Area : REAL);
```

```
VAR Pie : REAL;
```

```
BEGIN
```

```
  GetPi(Pie);
```

```
  Area := Pie * Radius * Radius;
```

```
END AreaOfCircle;
```

```
...
```

Exempel

```
class StackClass{
private int[] stackRef;
private int maxLen, topIndex;

public StackClass(){
    stackRef = new int[100]; maxIndex = 99;
    topIndex = -1;
}

public void push(int number){
    if(topIndex == maxIndex){
        // Error
    } else {
        stackRef[++topIndex] = number;
    }
}

public void pop(){
```

Exempel

```
exception EmptyStack;

abstype 'a stack = Stack of 'a list
with
  val empty = Stack []
  fun push(Stack s, e) = Stack(e::s)
  fun pop(Stack []) = raise EmptyStack
    | pop(Stack (_,l) ) = Stack l
  fun top(Stack [] ) = raise EmptyStack
    | top(Stack (a,_) ) = a
  fun isEmpty( Stack s ) = s = []
end;
```

Exempel

```
class Stack:
    def __init__(self):
        self.data = []

    def push(self,value):
        self.data.append(value)

    def pop(self):
        if self.data == []:
            raise "Empty Stack"
        self.data = self.data[:-1]

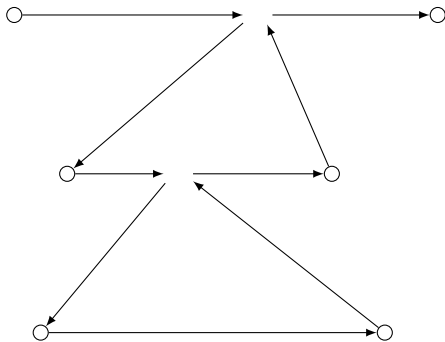
    def top(self):
        if self.data == []:
            raise "Empty Stack"
        return self.data[-1]

    def isEmpty(self): return self.data == []
```

- Återigen en abstraktion, men av kod i stället för data
- Procedurer, funktioner och metoder
 - Procedurer gör något (ändrar till exempel värden på variabler)
 - Funktioner returnerar ett värde
 - Metoder är “funktioner och procedurer för objekt”
- Vanligtvis definieras namn, parametrar och resultat

- Formella parametrar (formal parameters): Parametrarna i funktionens definition
- Faktiska parametrar (actual parameters): Parametrarna i funktionsanropet
- Bindningen av namn och värde kan gå till på olika sätt
- Godtyckligt antal
- Default-värden

“Normalt” beteende



Exempel

```
def example1(p1, p2):  
    print p1, p2  
example1(4,5) # 4 5
```

```
def example2(weight=20, length=57):  
    print weight, length  
example2(length=12) # 20 12
```

```
def example3(p1, weight=20):  
    print p1, weight  
example3(1) # 1 20
```

```
def example4(p1, *p2):  
    print p1  
    print p2  
example4(1) # 1 ()  
example4(1,2,3,4,5) # 1 (2,3,4,5)
```

Exempel

```
def example5(weight=20, length=57, **parameters):  
    print weight  
    print length  
    print parameters
```

```
example5(length=12,myname="Halvard Nattluva")
```

```
# 20, 12, {'myname': 'Halvard Nattluva'}
```

```
def example6(para, *manypara, **kwpara):  
    print para  
    print manypara  
    print kwpara
```

```
example6("Kalle","Anka","kalle@disney.com",engfirst="Donald",englast="")
```

```
# Kalle ('Anka', 'kalle@disney.com'), {'englast': 'Duck', 'engfirst': 'Donal
```

- Riktning
 - in-mode
 - out-mode
 - in-out-mode
- Huvudsakligen
 - värde
 - referens

Metoder för parameteröverföring

- Pass-by-value
- Pass-by-result
 - Möjligt problem: `procA(a, a)`
 - Möjligt problem: `procA(a[i], i)`
- Pass-by-value-result
 - Möjligt problem: `procA(4+4)`
- Pass-by-reference
 - Vanligt
 - “Öppnar upp” information

- Typkontroll: när och hur?
- Nästlade funktioner/procedurer
- Ska funktioner/procedurer kunna vara parametrar
- Överladdade funktioner/procedurer
- Generiska funktioner/procedurer
- Sidoeffekter
- Returvärden: vad kan returneras? hur många värden?

Exempel

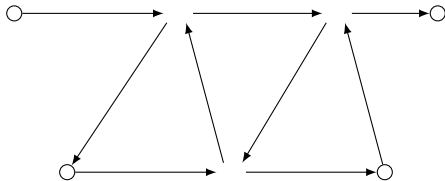
```
def example(a,b):  
    return a + b
```

```
example(1,2)
```

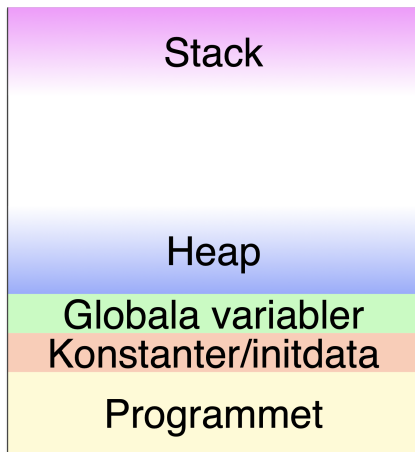
```
example("Hello", "World")
```

```
example([1,2,3],[4,5,6])
```

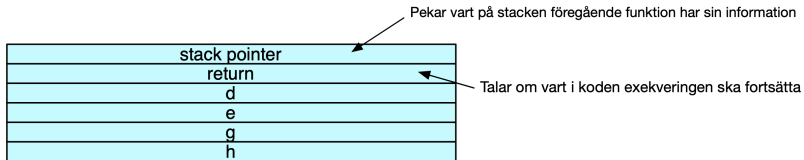
- Ser inte på samma sätt anropande och anropad funktion som “master” och “slave”.
- En procedur/funktion kan ha flera ingångsställen
- Kan användas för att få jämnlöpande program



Subprogram i minnet



Subprogram i minnet



Subprogram i minnet

